C Subset Definition

1. Overview

This document defines the subset of C supported by the compiler developed in Project 4. The language is a structured, statically-typed subset of C targeting LLVM IR as output. The compiler substantially exceeds the minimal required feature set; all sections below reflect what the grammar actually implements.

---

2. Lexical Elements

2.1 Keywords

```
int   float   double  void   char   bool
unsigned  long  short  const  static
if    else    while   for    do
return  break   continue  goto  sizeof
enum  struct  union  typedef  switch  case  default
NULL  true   false
```

2.2 Identifiers

A sequence of letters (`a–z`, `A–Z`), digits (`0–9`), and underscores (_), starting with a letter or underscore.

```
Identifier ::= [a-zA-Z_][a-zA-Z0-9_]*
```

2.3 Integer Constants

```
Integer_constant       ::= [0-9]+ [uUlL]?
HexInteger             ::= 0[xX][0-9a-fA-F]+ [uUlL]*
LongLongConstant       ::= [0-9]+ (u|U)?(l|L)(l|L)(u|U)?
                         | [0-9]+ (u|U)(l|L)(l|L)
CharLit                ::= '\'' ( EscapeSequence | any char except '\'' ) '\''
```

U/L/LL suffixes are accepted and ignored (all decimal integers are treated as `i32`; hex with the `LL` suffix becomes `i64`).

2.4 Floating-Point Constants

```
Floating_point_constant ::=
    [0-9]+ '.' [0-9]* ( [eE] [+-]? [0-9]+ )? [fF]?
  | [0-9]+             ( [eE] [+-]? [0-9]+ )  [fF]?
```

A trailing `f`/`F` suffix produces a `float` literal; otherwise the literal is `double`.

2.5 String Literals

```
STRING_LITERAL ::= '"' ( EscapeSequence | any char except '\' and '"' )* '"'
EscapeSequence ::= '\' ( 'b' | 't' | 'n' | 'f' | 'r' | '\'' | '"' | '\\' | '0' )
```

Adjacent string literals are automatically concatenated (C standard behaviour).

2.6 Operators

```
Arithmetic:       +   -   *   /   %   ##
Assignment:       =  +=  -=  *=  /=  %=  &=  |=  ^=  <<=  >>=
Increment:        ++  --
Comparison:       >   >=  <   <=  ==  !=
Logical:          &&  ||  !
Bitwise:          &   |   ^   ~   <<  >>
Ternary:          ? :
Address/deref:    &   *
Member access:    .   ->
```

2.7 Comments

```
/* block comment */
// line comment
```

2.8 Preprocessor

All `#include`, `#define`, `#if`, `#endif`, and other # directives are skipped at the lexer level; they do not affect code generation.

2.9 Whitespace

Spaces, tabs, and newlines are ignored.

---

3. Data Types

Type	Description	LLVM IR type
`int`	32-bit signed integer	`i32`
`float`	32-bit IEEE 754 floating-point	`float`
`double`	64-bit IEEE 754 floating-point	`double`
`void`	No value; function return type only	`void`
`char`	8-bit integer / byte	`i8`
`bool`	Boolean (1-bit)	`i1`
`long long` / `unsigned long long`	64-bit integer	`i64`
`unsigned`, `long`, `short`, `unsigned int`, `unsigned long`	Mapped to `i32`	`i32`
Pointer `T *`	Pointer to T	`ptr`
Array `T[N]`	Stack-allocated array	`[N x T]`
Multi-dim `T[N][M]`	Nested arrays	`[N x [M x T]]`
`struct Name`	User-defined struct (with named fields)	`%struct.Name`

Type	Description	LLVM IR type
`union Name`	User-defined union (treated as struct)	`%struct.Name`
`typedef` alias	Alias for any of the above	same as aliased type

---

4. Program Structure

```
program ::= ( enum_def | struct_def | union_def | typedef_def
              | global_var_decl | func_proto | func_def )+
```

Execution starts at `main`. Functions may accept `int argc, char* argv[]` or no parameters.

4.1 Function Definition

```
func_def ::= type ptr_stars identifier '(' params ')' '{' declarations
statements '}'
params   ::= ( type ptr_stars identifier ('[' ']')? (',' type ptr_stars
identifier ('[' ']')?)* )?
```

- Return type may be any scalar type, pointer, or `void`.
- Parameters may be scalar, pointer, or array-decayed pointer (`T[]`).
- Function signatures are registered before their body is parsed; forward calls are supported within the same file provided the callee appears before the caller. Function prototypes (`;`-terminated signatures) support mutual recursion.

4.2 Function Prototypes

```
func_proto ::= type ptr_stars identifier '(' params ')' ';'
```

Registers the function signature without emitting code. Enables mutual recursion.

---

5. Global Declarations

5.1 Global Variables

```
global_var_decl ::= [const | static]* type ptr_stars name ['=' expr] (','
ptr_stars name ['=' expr])* ';'
                  | [const | static]* type name dim_list ';'   // global array
```

- Global scalars are emitted as LLVM global variables initialised to zero.
- Global arrays are zero-initialised (`zeroinitializer`).
- Initialiser expressions on globals are parsed but initial values are set to zero in the emitted IR (dynamic initialisation is not supported).

5.2 Enums

```
enum_def ::= 'enum' identifier? '{' name ['=' expr] (',' name ['=' expr])* ','?
'}' ';'
```

- Enum constants are stored in a global table and substituted as integer literals.
- Values may be integer expressions (including hex literals and bit-shifts).
- Unnamed enums are permitted.

5.3 Structs and Unions

```
struct_def ::= 'struct' name '{' struct_field+ '}' ';'
union_def  ::= 'union'  name '{' struct_field+ '}' ';'
struct_field ::= type ptr_stars name ';'              // plain field
               | type name ':' integer ';'            // named bit-field (width
ignored)
               | type ':' integer ';'                 // unnamed padding
(ignored)
               | 'union'  '{' struct_field+ '}' ';'  // anonymous union  (fields
flattened)
               | 'struct' '{' struct_field+ '}' ';'  // anonymous struct (fields
flattened)
```

Unions are emitted as structs (only the field list is retained; the union overlay semantics are not enforced at the IR level). Anonymous union and struct blocks are flattened into the enclosing struct. Bit-field width is accepted syntactically but ignored.

5.4 Typedefs

```
typedef_def ::= 'typedef' type ptr_stars alias_name ';'
              | 'typedef' type ptr_stars '(' '*' fp_name ')' '(' type_list ')'
';'
```

Supports scalar typedefs and function-pointer typedefs. The alias is recorded and resolved during type parsing.

---

6. Local Declarations

All local variables must be declared at the top of the function body before the first statement (C89 style). Additionally, the compiler supports C99-style declarations interleaved with statements inside any block.

```
declarations ::= struct_decl ';' declarations
               | char name '[' ']' '=' STRING_LITERAL+ ';' declarations   //
char array from string
               | [const|static]? type name dim_list ['=' '{' init_list '}'] ';'
declarations
               | [const|static]? type ptr_stars name ['=' expr] (',' ptr_stars
name ['=' expr])* ';' declarations
               | ε
```

- Variables are stack-allocated (`alloca`).
- Array initialisers (`= { ... }`) are parsed; values are consumed but not individually

emitted — the array is zero-initialised via `memset`.

- Designated initialisers (`[N] = val`, `.field = val`) are accepted syntactically but ignored for code generation.
- `char name[] = "string"` allocates a properly sized buffer and copies the string constant.
- Variables may be initialised at declaration: `int x = expr;`.

---

7. Statements

```
statement ::= assign_stmt ';'
            | inc_dec_stmt ';'
            | if_stmt
            | while_stmt
            | for_stmt
            | do_while_stmt
            | switch_stmt
            | return_stmt ';'
            | func_call_stmt ';'
            | break_stmt
            | continue_stmt
            | goto_stmt
            | labeled_stmt
            | block_stmt
            | inline_decl ';'   // C99 mid-block declaration
            | ';'               // empty statement
```

7.1 Assignment Statement

```
assign_stmt ::= lvalue op rhs
lvalue      ::= identifier                    // scalar variable
              | identifier '[' expr ']' ...   // array element (any number of
dimensions)
              | identifier '.' field          // struct field
              | identifier '->' field         // pointer-to-struct field
              | '*' identifier                // dereference
op          ::= '=' | '+=' | '-=' | '*=' | '/=' | '%=' | '&=' | '|=' | '^=' |
'<<=' | '>>='
```

All compound operators (`+=`, `&=`, `<<=`, etc.) load the current value, apply the operation, and store the result.

7.2 Increment / Decrement Statement

```
inc_dec_stmt ::= identifier '++' | identifier '--'
               | '++' identifier | '--' identifier
```

Both prefix and postfix forms are supported. When used as a statement, both behave identically.

7.3 `if` Statement

```
if_stmt ::= 'if' '(' expr ')' statement ( 'else' statement )?
```

- The condition is any expression (compared `!= 0`) or a relational expression.

- Braces are not mandatory (bare single-statement bodies are accepted).
- `if`/`else` chains may be nested to any depth.

7.4 `while` Statement

```
while_stmt ::= 'while' '(' expr ')' statement
```

Braces are not mandatory. `break` and `continue` are supported inside.

7.5 `for` Statement

```
for_stmt ::= 'for' '(' for_init ';' [expr] ';' for_update ')' statement
for_init  ::= assign_stmt | inc_dec_stmt | for_decl | ε
for_update ::= (assign_stmt | inc_dec_stmt) (',' (assign_stmt | inc_dec_stmt))*
for_decl   ::= type name ['=' expr] (',' name ['=' expr])*
```

- C99-style declarations in the initialiser (`for (int i = 0; ...)`) are supported.
- Multiple update expressions separated by commas are supported.
- `break` and `continue` are supported inside.

7.6 `do-while` Statement

```
do_while_stmt ::= 'do' statement 'while' '(' expr ')' ';'
```

`break` and `continue` are supported inside.

7.7 `switch` Statement

```
switch_stmt ::= 'switch' '(' expr ')' '{' case_clause* default_clause? '}'
case_clause   ::= 'case' case_val ':' statement*
default_clause ::= 'default' ':' statement*
case_val       ::= integer_constant | hex_constant | char_literal |
enum_identifier | '-' integer
```

- C-style fall-through is supported (the absence of `break` falls to the next case).
- `default` is optional.
- Case values may be integer constants, hex constants, character literals, or enum identifiers.
- `break` exits the switch.

7.8 `return` Statement

```
return_stmt ::= 'return' expr
              | 'return'
```

`return expr` coerces the expression to the function's declared return type. `return` (no value) is for `void` functions. `main` always returns `i32`.

7.9 `break` / `continue`

`break` jumps to the end of the nearest enclosing `while`, `for`, `do-while`, or `switch`.

`continue` jumps to the condition/update of the nearest enclosing loop.

7.10 `goto` and Labels

```
goto_stmt    ::= 'goto' identifier ';'
labeled_stmt ::= identifier ':' statement
```

Labels are resolved lazily (a `goto` may target a label defined later in the function). Both forward and backward jumps are supported.

7.11 Function Call Statement

```
func_call_stmt ::= identifier '(' call_args ')'
```

Used for `printf`, `scanf`, void user-defined calls, and value-returning calls whose result is discarded.

7.12 Block Statement

```
block_stmt ::= '{' statements '}'
```

---

8. Expressions

8.1 Precedence (high to low)

Level	Operators	Associativity
1 (highest)	Unary `-`, `+`, `~`, `!`, `&` (addr-of), `*` (deref)	Right
2	Type cast `(type)`	Right
3	`* / % ##`	Left
4	`+ -`	Left
5	`<< >>`	Left
6	`> >= < <= == !=`	Left
7	`&` (bitwise AND)	Left
8	`^`	Left
9	`|`	Left
10	`&&`	Left
11	`||`	Left
12 (lowest)	`? :` (ternary)	Right

8.2 Primary Expressions

```
primary ::= integer_constant
          | hex_constant
```

```
           | long_long_constant
           | char_literal
           | floating_point_constant
           | string_literal
           | NULL | true | false
           | identifier
           | identifier '(' call_args ')'           // function call
           | identifier '[' expr ']' ...            // array element read
           | identifier '[' expr ']' '(' args ')'  // function-pointer array
call
           | identifier '.' field                  // struct member read
           | identifier '->' field                 // pointer-to-struct member
read
           | sizeof '(' type dim_list? ')'
           | sizeof '(' 'struct' name ')'
           | '(' expr ')'
           | '(' comma_expr ')'                    // comma operator in parens
```

8.3 Logical Operators

`&&` and `||` are supported with short-circuit semantics (emitted as `and i1` / `or i1` on truthiness values). `!` (logical NOT) flips the truthiness bit.

8.4 Bitwise Operators

`&`, `|`, `^`, `~`, `<<`, `>>` are supported for integer operands. All are constant-folded when both operands are compile-time constants.

8.5 Ternary Operator

```
ternary ::= expr '?' expr ':' ternary
```

Emits a stack slot and two branches to implement the conditional. Result type is the wider of the two branches.

8.6 Comma Operator

Inside parentheses: `(a, b, c)` evaluates each expression left-to-right and yields the last value. Used in `for` update clauses and complex expressions.

8.7 Address-of and Dereference

- `&identifier` — yields the stack-slot address as a `ptr` value.
- `*expr` — loads through a pointer (or decays an array to its first element).

8.8 Type Cast

```
cast_expr ::= '(' type ptr_stars ')' expr
            | '(' 'struct' name ptr_stars ')' expr
```

Scalar casts: `(int)`, `(float)`, `(double)`, `(char)`, pointer casts. All are emitted as appropriate LLVM conversion instructions or as opaque pointer casts.

8.9 `sizeof`

```
sizeof '(' type dim_list? ')'     // e.g. sizeof(int), sizeof(int[4][3])
sizeof '(' 'struct' name ')'      // sum of field sizes (naïve, no padding)
```

Evaluated at compile time; emits an integer constant.

8.10 The ## Operator

```
a ## b
```

Defined as **a^b + b^a**, where both operands are coerced to `float`. Result type is always `float`. Precedence equals `*` and `/`. Constant-folded at compile time when both operands are known. At runtime, implemented via the `__hashOp` runtime function.

---

9. Type System

9.1 Implicit Type Conversion

Situation	**Behaviour**
`int op double` or `double op int`	`int` promoted to `double`; result is `double`
`int op float` or `float op int`	`int` promoted to `float`; result is `float`
`float op double`	`float` promoted to `double`; result is `double`
Assign `float → int`	Truncation towards zero (`fptosi`)
Assign `int → float`	Exact conversion (`sitofp`)
Assign `int → double`	Exact conversion (`sitofp`)
`bool → int`	Zero-extension (`zext`)
`int → bool`	`icmp ne i32 x, 0`
`char → int`	Sign-extension (`sext`)
`int → i64`	Sign-extension (`sext`)
`i64 → int`	Truncation (`trunc`)
`float` argument to `printf`	Promoted to `double` (C default argument promotion)
Pointer arithmetic (`ptr ± int`)	GEP instruction

9.2 Explicit Type Cast

```
(int)   expr    // truncates float/double to int, or no-op for int
(float) expr    // converts int/double to float, or no-op for float
(double)expr    // converts int/float to double, or no-op for double
(char*) expr    // opaque pointer cast
```

9.3 Type Errors

The following terminate compilation with an error message and line number:

- Reading or writing an **undeclared identifier**
- **Re-declaring** an identifier already declared in the same function

---

10. Built-in / Runtime Functions

10.1 `printf`

```
printf(STRING_LITERAL, arg1, arg2, ...)
```

Supported format specifiers: `%d` (int), `%f` (float/double), `%s` (string), `%x` (hex), and any others passed through to the C runtime. `float` arguments are promoted to `double` per C convention.

10.2 `scanf`

```
scanf(STRING_LITERAL, &var1, &var2, ...)
```

Supported format specifiers: `%d` (reads into `int`), `%f` (reads into `float`), `%s` (reads into `char` array). Arguments must be address-of expressions.

10.3 Standard library functions (declared, not wrapped)

The following are declared in the IR preamble and usable via normal call syntax:

Function	Signature
`malloc`	`ptr malloc(i64)`
`free`	`void free(ptr)`
`strcpy`	`ptr strcpy(ptr, ptr)`
`strcat`	`ptr strcat(ptr, ptr)`
`strlen`	`i32 strlen(ptr)`
`strcmp`	`i32 strcmp(ptr, ptr)`
`memcpy`	`ptr memcpy(ptr, ptr, i64)`
`memset`	`ptr memset(ptr, i32, i64)`
`abs`	`i32 abs(i32)`
`fabs`	`double fabs(double)`
`sqrt`	`double sqrt(double)`
`pow`	`double pow(double, double)`

10.4 `__hash0p` (runtime library)

```
float __hash0p(float a, float b);   // = pow(a,b) + pow(b,a)
```

Implemented in `myRuntime.c`. Linked at the `clang` step.

---

11. Scoping Rules

- All variables are **function-scoped** (variables declared anywhere in a function are visible throughout the rest of the function body).
- No block scoping: a variable declared inside an `if` or `while` body is visible after the block ends.
- Global variables are visible in all functions.
- Function names are globally visible throughout the file.
- Enum constants and struct/union/typedef names are global.

---

12. Code Optimizations

The compiler performs several optimisations during code generation (no separate optimisation pass needed).

12.1 Constant Folding

Arithmetic expressions whose operands are both compile-time constants are evaluated at compile time — no instruction is emitted.

```
a = 2 + 3;      // emits: store i32 5
x = 1.5 * 2.0;  // emits: store double <hex for 3.0>
y = 2.0 ## 3.0; // emits: store float <hex for 17.0>
```

Works for `+`, `-`, `*`, `/`, `%`, `&`, `|`, `^`, `<<`, `>>`, `~`, unary minus, ##, type conversions, and comparisons.

12.2 Algebraic Identity Simplifications

Trivially redundant operations are eliminated without emitting instructions:

Expression	Simplified to
`x + 0`, `x - 0`	`x`
`x * 1`, `x / 1`	`x`
`x * 0`	`0`

These apply to `int`, `float`, and `double`, with correct type promotion.

12.3 Constant Propagation

When a variable is assigned a constant, subsequent reads are replaced with the constant — eliminating `load` instructions entirely.

```
a = 10;      // constSlot[a] = 10
```

```
b = a + 5;   // reads 10 from constSlot → folds to 15, no load emitted
c = b * 2;   // reads 15 → folds to 30, no load emitted
```

The propagation table is conservatively invalidated at all control-flow merge points (after `if`/`else`, at loop condition labels, after `while`/`for`/`do-while`/`switch`), and at `scanf` calls that write through `&var` pointers.

12.4 Constant Type Conversion

Coercions between `int`, `float`, and `double` constants are resolved at compile time without emitting conversion instructions.

```
float x = 5;     // emits: store float 0x4014000000000000  (no sitofp)
int   n = 2.9;   // emits: store i32 2  (no fptosi)
```

---

13. Example Programs

Minimal program

```
int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Mixed types, while loop, functions

```
float average(int a, int b) {
    float sum;
    sum = (float)a + (float)b;
    return sum / 2.0;
}

int main() {
    int i, n;
    float acc;
    acc = 0.0;
    n = 5;
    i = 1;
    while (i <= n) {
        acc += i;
        i++;
    }
    printf("Sum = %f, Average = %f\n", acc, average(1, n));
    return 0;
}
```

Enum, struct, for loop, switch

```
enum Day { MON, TUE, WED };

struct Point { int x; int y; };

int main() {
    struct Point p;
    int d, i;
    p.x = 3;
```

```
    p.y = 4;
    printf("(%d, %d)\n", p.x, p.y);

    d = WED;
    switch (d) {
        case MON: printf("Monday\n");    break;
        case WED: printf("Wednesday\n"); break;
        default:  printf("Other\n");     break;
    }

    for (i = 0; i < 3; i++) {
        printf("%d\n", i);
    }
    return 0;
}
```

---

14. Features NOT Supported

The following standard C features are either silently accepted but not fully code-generated, or are outside the scope of this compiler:

- Struct initialisation from `= { ... }` syntax (parsed but zero-initialised; individual field values are not set at compile time)
- Global variable initialisers (globals are always zero-initialised in the emitted IR)
- Variable-length arrays (VLAs)
- `_Bool`, `_Complex`, `_Generic`
- `register`, `volatile`, `extern`, `inline` storage/qualifier keywords
- Recursive struct types (linked lists, trees)
- Variadic user-defined functions (`...`)